CURS 8 PAA

GRAFURI

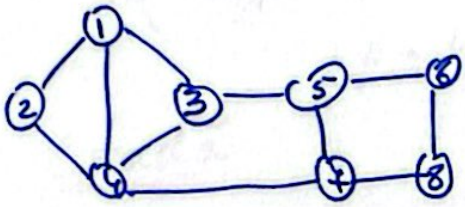
N - mult. noduri

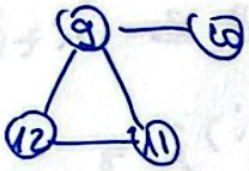
A - arce

→ concept de vecini în loc de rădăcină / copii

- neponderate (se poate asocia ponderea 1 pt. a aplica alg. de la ponderate)
- ponderate (cu val. pe arce)

- neorientate (se pot aplica și alg. de la orientate)
- orientate

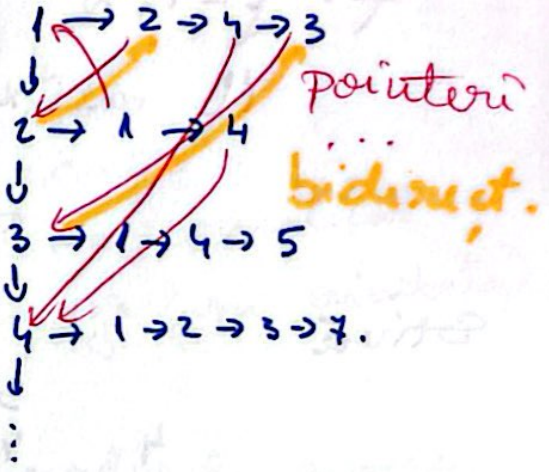




13.

Structuri de adiacență

*rar



graf rar:

len(N) >> len(A)

ex: 10.000 noduri
5 arce

Matrici de adiacență * cea mai folosită

a[i][j] = 1 dacă i → j

a[i][j] = a[j][i] (neorientat)

	1	2	3	4	5	6	7	8	9	10	11	12	13
1	0	1	1	1	0	0	0	0	0	0	0	0	0
2	1	0	0	1	0	0	0	0	0	0	0	0	0
3	1	0	0										
4	1	1		0									
5	0	0			0								
6	0	0				0							
⋮													

$O(n^2)$

a[i][j] = p i → j pt. arbori ponderați.

Scanned with CamScanner

pt. examen : → se poate da matr.
→ se pot da i, j; (P)

Parcurgeri prim vecin ne vizitat – recursiv

1 2 4 3 5 6 8 7 9 10 11
12 13. cc = 0 ← pt. a afla câte comp. conexe.

```
for i = 1, N
    if viz[i] == 0
        parc_adânc(i) / parc_cuprind(i).
                                cc ++;
```

```
parc_adânc (Nod x)
    for j = 1, N
        if a[x][j] != 0 && viz[j] == 0
            parc_adânc (j);
```

(7) nu mai are alt vecin (stergem)

Stivă

			6	6	6	6
		5	5	5	5	5
		3 3	3	3	3	3
	4	4 4	4	4	4	4
1	2	2 2	2	2	2	2
	1	1 1	1	1	1	1

→ până stiva devine goală

```
while stiva nu e goala
    top
    for i = 1, n
        if a[top][i] && viz[i] == 0
            push(i)
    pop();
```

1 2 3 4 5 7 6 8

9 10 11 12

13.

```
while (coadă nu e goală)
    top, pop()
    for (i=1, n)
        if (a[top][i] != 0 && viz[i] == 0)
            push(i)
```

Coadă

1

2 3 4

3 4

4 5

5 7

7 6

6 8

8

4

9

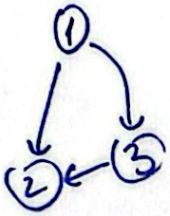
10 11 12

11 12

12

1

13

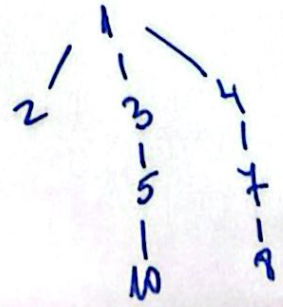


3 comp. conexe
(e mai complicat la grafuri orientate)

comp. conexă
→ din toate nodurile ajungem în toate

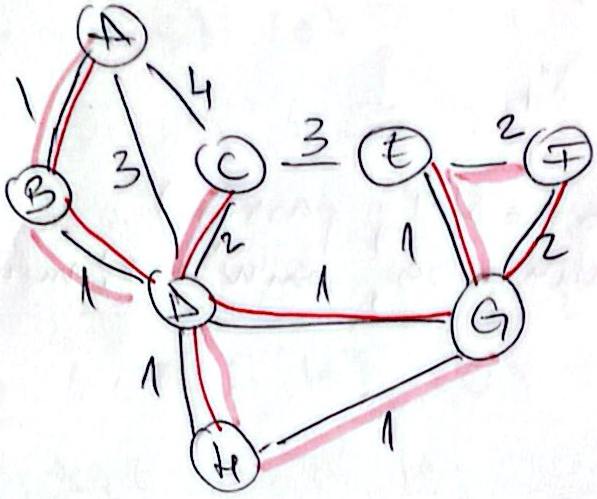
{ N noduri
N-1 arce

Arbore de acoperire



CURS 5 PAA

Grafuri ponderate



- arbore de acoperire minim => suma arcelor e minimă (dacă se cere cost. minim pt. toate ...).
- algoritmul lui prim & algoritmul lui Kruskal

PRIM

N - noduri
N-1 arce

U	N-U	↓ alegem drum minim
A	B C D E F G H	A-B
A B	C D E F G H	B-D
A B D	C E F G H	D-G.
A B D G	C E F H.	D-H,
A B D G H	C E F	G-E
A B D E G H	C F	D-C
A B C D E G H	F	G-F
D B C D E F G H.	—	

→ ne oprim după n-1 pași

int S[10];
S[i] = 1 $i \in U$.
0 $i \notin U$.
} vector de apariții.

```
void prim(A, N)
{ int S[10] = {0};
  S[0] = 1;
  for(pas = 1; pas <= N-1; pas++)
  {  min = 99; mini = -1; minj = -1;
     for(i
        for j
           if(A[i][j] != 0 && S[i] != S[j] &&
              min > A[i][j])
              min = A[i][j];
              mini = i;
              minj = j;
     printf("%d->%d", mini, minj);
     S[mini] = S[minj] = 1;
  }
}
```

$O(n^3)$

Scanned with CamScanner

Kruskal

alegem cele mai mici

{A} {B} {C} {D} {E} {F} {G} {H} — G-H.

{A} {B} {C} {D} {E} {F} {G, H} — B-D.

{A} {B, D} {C} {E} {F} {G, H} — D-H

{A} {B, D, G, H} {C} {E} {F} — A-B

{A, B, D, G, H} {C} {E} {F}. — G-E

{A, B, D, E, G, H} {C} {E} — E-F.

{A, B, D, E, F, G, H} {C} — D-C

{A, B, C, D, E, F, G, H}

n-1 pași.

```
int S[10]
S[i] = x  i ∈ V.
S[i] = i.
void KruskalA, N)
 int S[10] = {0};
   for i = 0, N      S[i] = i
   for pas = 1, N-1
            min = 99; mini = -1; minj = -1;

   for i
      for j
         if(A[i][j] != 0 && S[i] != S[j]
         && min > A[i][j]).
            min = A[i][j]; mini = i; minj = j;
```


Scanned with CamScanner

```
for i = 0,N
    if(S[i]==S[min j])
        S[i] = S[min i];
```

se pot aplica și pe Grafuri ne ponderate

PRIM & KRUSKAL la examen

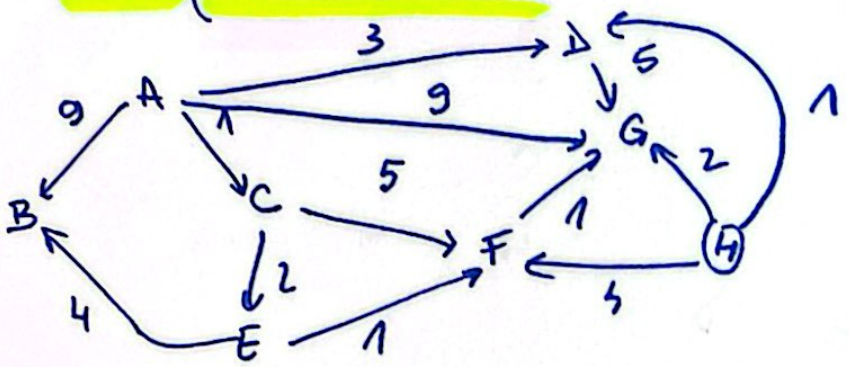
DIJKSTRA + FLOYD
└ nu prea se pocate la examen

- se pocate cere să aflăm toți arborii de acoperire minim.

Scanned with CamScanner

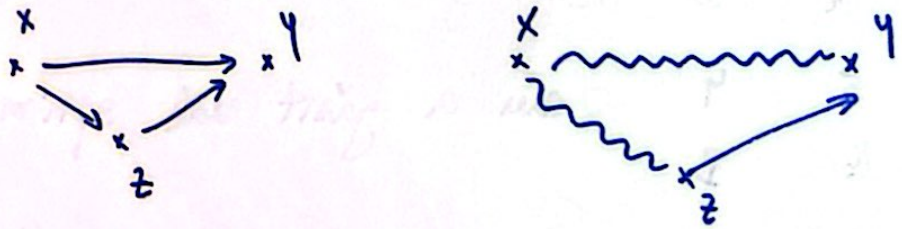
Curs 10 PAA

Grafuri orientate



Dijkstra → este greedy

$O(v^2)$



$\exists z$ a.î. $D(x,y) > D(x,z) + A[z][y]$.

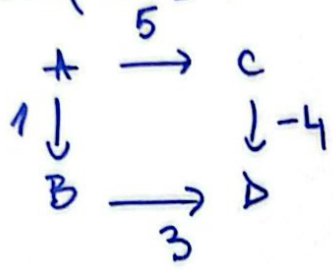
$D(x,y) \leftarrow D(x,z) + A[z][y]$.

	A	B	C	D	E	F	G	H	
D	-	9	1	3	∞	∞	9	∞	← lung. drum
T	A	A	A	A	A	A	A	A	← drumul prin
D	-	9	1	3	3(1+2)	6(1+5)	9	∞	
T	A	A	A	A	C	C	A	A	
D	-	9	1	3	3	6	8(5+3)	∞	
T	A	A	A	A	C	C	D	H	
D	-	7(3+4)	1	3	3	4(3+1)	8	∞	
T	A	E	A	A	C	E	D	A	
D	-	7	1	3	3	4	5(1+4)	∞	
T	A	E	A	A	C	E	F	A	
D	-	7	1	3	3	4	5	∞	
T	A	E	A	A	C	E	F	A	
D	-	7	1	3	3	4	5	∞	m-1 pași
T	A	E	A	A	C	E	F	A.	

Drum $A \to G$

$A \xrightarrow{1} C \xrightarrow{2} E \xrightarrow{1} F \xrightarrow{1} G \Rightarrow 5$

Exemplu 2.



	A	B	C	D
Δ	-	1	5	∞
T	A	A	A	A
Δ	-	1	5	4 (1+3)
T	A	A	A	B
Δ	-	1	5	4
T	A	A	A	B

nu a găsit val. optimă

Floyd Marshall → programare dinamică

$O(n^3)$

```
for k
    for i
        for j
if D[i,j] > D[i,k] + A[k][j];
        D(i,j) = D[i,k] + A[k][j];
```

```
D[i][j] = A[i][j];
```

	A	B	C	D
A	0	1	5	∞
B	∞	0	∞	3
C	∞	∞	0	-4
D	∞	∞	∞	0

k = A;

	A	B	C	D
A	0	1	5	∞
B	∞	0	∞	3
C	∞	∞	0	-4
D	∞	∞	∞	0

Scanned with CamScanner

$k = B$

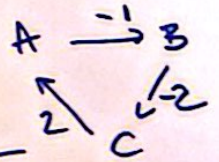
	A	B	C	D
A	0	1	5	4
B	∞	0	∞	3
C	∞	∞	0	-4
D	∞	∞	∞	0

$k = C$

	A	B	C	D
A	0	1	5	1
B	∞	0	∞	3
C	∞	∞	0	-4
D	∞	∞	∞	0

$k = D$

	A	B	C	D
A	0	1	5	1
B	∞	0	∞	3
C	∞	∞	0	-4
D	∞	∞	∞	0



Eroare la Floyd
=> nu e voie cu cicluri neg.

TEST.

Grafuri → parc. + comp. conex. – mai puțin
↳ arb. de acop. min | drum minim – mai mult.

Scanned with CamScanner